

TP7 – Validation JSON, Versioning et S rialisation

Aymane Es-salehy

21 avril 2026

1 Introduction

L’objectif de ce TP est de concevoir un syst me robuste de validation et de s rialisation des donn es en utilisant JSON et Protobuf. Nous avons impl ment  des m canismes de validation stricte, de versioning et  tudi  les implications de s curit .

2 TP7.1 – Contrat JSON et validation

2.1 Description

Deux entit s ont  t  d finies :

- Document
- UserPublic

Chaque champ est valid  selon :

- Type
- Longueur
- Allowlist (classification, r le)
- Format (date ISO 8601)

Une approche **fail closed** a  t  utilis e : toute donn e invalide est rejet e.

2.2 Résultats des tests

```
aymane@aymane-VirtualBox:~/TP7$ python3 tp7_validation_json.py
===== TEST 1 : VALID DOCUMENT =====
Document(id=1, title='Rapport', author='Alice', tags=[], classification='internal', created_at='')

===== TEST 2 : VALID USER =====
UserPublic(username='alice_01', display_name='Alice', role='editor')

===== TEST 3 : MISSING FIELD =====
[LOG] Missing field: id
Invalid document payload

===== TEST 4 : WRONG TYPE =====
[LOG] Invalid id
Invalid document payload

===== TEST 5 : INVALID ROLE =====
[LOG] Invalid role
Invalid user payload
```

FIGURE 1 – Résultats des tests de validation

2.3 Analyse

- Les données valides sont correctement désérialisées
- Les erreurs sont détectées avec précision
- Le système ne laisse passer aucune donnée invalide

3 TP7.2 – Versioning JSON

3.1 Principe

Le système a été étendu de v1 à v2 avec ajout de champs :

- tags
- classification

Les anciennes versions sont supportées via des valeurs par défaut.

3.2 Résultats des tests

```
===== TEST V1 → V2 =====
Document(id=1, title='Doc', author='Alice', tags=[], classification='internal', created_at='')

===== TEST V2 COMPLET =====
Document(id=2, title='Doc2', author='Bob', tags=['ai'], classification='public', created_at='')

===== TEST UNKNOWN FIELD =====
[LOG V2] Unknown field: priority
Invalid document payload (v2)
```

FIGURE 2 – Résultats des tests de validation

3.3 Matrice de compatibilité

Version	Lecteur	Accepté	Raison	Risque
v1	v2	Oui	Valeurs par défaut	Aucun
v2	v1	Partiel	Champs ignorés	Perte d'information
v2 altéré	v2	Non	Allowlist violée	Risque sécurité
v2 + inconnu	v2	Non	Fail closed	Injection possible

4 TP7.3 – Protobuf

4.1 Principe

Protobuf permet une sérialisation binaire compacte et typée.

4.2 Comparaison

Critère	JSON	Protobuf	Commentaire
Taille	Grande	Petite	Protobuf est plus compact
Lisibilité	Élevée	Faible	JSON lisible
Validation	Manuelle	Automatique	Schéma strict
Compatibilité	Moyenne	Excellente	Ajout de champs facile
Tooling	Simple	Requiert protoc	

5 TP7.4 – Politique de sérialisation

5.1 Tableau

Source	Format	Contrôles	Justification
API REST	JSON	Validation stricte	Données non fiables
Inter-services	JSON/Protobuf	mTLS + validation	Zero trust
Cache local	pickle	Local uniquement	Données sûres
Upload utilisateur	JSON/CSV	Validation	Sécurité
Message queue	JSON/Protobuf	Signature	Multi sources
Stockage	JSON/Protobuf	Chiffrement	Intégrité

5.2 Checklist de sécurité

- Interdire pickle pour les entrées externes
- Valider tous les champs
- Utiliser des allowlists
- Limiter la taille des payloads
- Logger les erreurs
- Signer les données (HMAC)
- Versionner les APIs
- Tester les cas limites
- Documenter les formats

6 Conclusion

Ce TP met en évidence l'importance de la validation stricte et du versioning dans les systèmes distribués. L'utilisation de Protobuf améliore les performances tandis que des politiques de sérialisation strictes renforcent la sécurité globale.